Since orchestrators provide the platform for running our apps, they are central to releasing new software (release management), knowing and managing how our cluster is using its resources (configuration management), and tasks like restarting services or scaling to match demand (operating).

What's nice about these tools is that they're declarative rather than imperative. Instead of our having to give the orchestrator a list of instructions that, if it follows, will achieve our desired state, we simply specify the desired state, and the orchestrator figures out how to achieve and maintain it. This allows us to think at a higher level, with the orchestrator handling many of the uninteresting, low-level details for us.

Because orchestrators understand the desired state, they are able to be smarter and more resilient to failure. If the code running in one of the containers crashes for some reason, the orchestrator can automatically restart it. If an entire node in the cluster fails, meaning all the containers it had been running are gone, again the orchestrator can recover and start up new replacement containers on the remaining nodes.

When you want to release a new version of your software, you tell the orchestrator that the desired state now uses your newer version of an image, and the orchestrator gets to work updating it. Orchestrators therefore play a key role in deployment and provide capabilities to perform rolling updates to help achieve zero-downtime deploys.

Orchestrators can help in other ways, too. Providing keys and sensitive configuration to apps is an age-old problem. However, in order to function, orchestrators need a secure, distributed way of managing the internal state of the cluster and communicating parts of it to nodes and containers on a need-to-know basis. That sounds suspiciously similar to distributed secret management. It's no surprise then that orchestrators expose secret management as an application-level feature.

Finally, as orchestrators play such a critical role at the heart of your production environment, they tend to have highly sophisticated security features—such as automatic key rotation—built in.

A Tale of Two Orchestrators: Swarm and Kubernetes

There are currently two competing orchestration layers that can be used with Docker: Swarm and Kubernetes. Both have many similarities but also significant differences.